

SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing

COURSE CODE: CSA37

COURSE OUTCOME COVERED

CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Code of Conduct:

I _P.Jyothika Reddy(192311156) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

P.Jyothika Reddy

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and wellorganized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1. Question

A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.

Answer:

Testing should be planned and performed as a systematic engineering activity.

Define test objectives, requirements, risks, and expected results before execution.

Use techniques such as equivalence partitioning, boundary value analysis, and decision tables.

Prepare traceable test cases covering normal, invalid, and edge-case inputs.

Review and prioritize tests based on risk and defect history.

This structured approach gives consistent coverage and detects more defects early.

2. Question

A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.

Answer:

A defined testing process provides clear activities, responsibilities, and entry and exit criteria.

Requirements are reviewed first, followed by test planning, design, execution, and reporting.

Critical banking functions such as login, withdrawal, transfer, and balance checks are prioritized.

Regression and security testing ensure that existing functions remain reliable after changes.

Defects are documented, tracked, fixed, and retested using a common process.

This improves reliability, traceability, compliance, and customer confidence in the banking system.

3. Question

During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.

Answer:

Testing should be planned from requirements through maintenance instead of being done only at the end.

Review requirements and design test conditions before coding begins.

Perform unit testing, integration testing, system testing, and acceptance testing at suitable stages.

Maintain test cases, expected results, defect reports, and traceability throughout the SDLC.

Use regression testing whenever modules such as cart, payment, or order processing are changed. Process discipline gives consistent coverage and reduces integration and system-level defects.

4. Question

A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.

Answer:

An error is a human mistake made during requirements, design, coding, or another activity.

A defect is a flaw in the software caused by an error and capable of producing incorrect behavior.

A failure occurs when the software executes and produces an incorrect or unexpected result.

For example, a developer may misunderstand the password rule, creating a validation defect.

When the login accepts an invalid password because of that defect, the observed mismatch is a failure. Thus, an error can introduce a defect, and the defect can cause a failure during execution.

5. Question

A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.

Answer:

Apply the principle that early testing saves time and cost.

Testers should review requirements, specifications, and designs before implementation.

Ambiguous rules can be identified and corrected before they become coding defects.

For example, an unclear refund rule can be clarified during requirement review.

This prevents incorrect implementation and reduces expensive system-level rework. Early detection improves quality because defects are removed closer to their source.

6. Question

During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.

Answer:

Testers should review requirements for ambiguity, completeness, consistency, and testability.

They can ask questions about unclear medical rules, inputs, outputs, and exceptional conditions.

Acceptance criteria and measurable expected results should be defined for important requirements.

Traceability can connect each requirement to test conditions and later test cases.

Early reviews prevent misunderstandings from being converted into implementation defects.

Tester involvement from the requirement phase therefore improves quality and reduces rework.

7. Question

A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.

Answer:

Defects can originate when requirements are incomplete, ambiguous, inconsistent, or incorrectly understood.

An unclear salary calculation rule may be interpreted differently by developers and testers.

The misunderstanding becomes a design or coding defect during later lifecycle stages.

Testing may reveal the incorrect salary output, but the original cause is the unclear requirement.

Clear definitions, examples, business rules, and acceptance criteria prevent such misunderstandings. Requirement reviews and traceability help identify and remove defects at their source.

8. Question

A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue. Answer:

- This is a functional or input-validation defect because the system handles valid boundary input incorrectly.
- Boundary values are important because errors often occur at minimum and maximum limits.
- For an age field allowing 18 to 60, values 18 and 60 must be explicitly tested.
- Values just below and above the limits, such as 17 and 61, should also be tested.
- Boundary Value Analysis systematically selects these edge cases for execution.
- It could have exposed the crash before release and improved input-handling reliability.

9. Question

A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.

Answer:

A defect repository stores details such as defect type, cause, severity, module, and resolution. Testers can analyze historical defects to identify frequently failing areas and patterns. Previous defects can be converted into regression and negative test cases. For example, repeated payment failures can lead to additional transaction and timeout tests. Repository data also helps prioritize high-risk modules and improve test coverage. Using historical defects prevents recurrence and continuously improves software quality.

10. Question

An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.

Answer:

- The problem is a functional defect because the withdrawal feature does not perform its required transaction behavior correctly.
- Examples include deducting the wrong amount, allowing withdrawal beyond balance, or failing to update the account.
- Test cases should cover sufficient balance, insufficient balance, invalid amounts, and transaction limits.
- Integration tests should also verify communication between the ATM, account, and transaction systems.
- Boundary and negative testing can expose incorrect transaction conditions.
- Thorough functional and regression testing could have prevented the defect from reaching users.

11. Question

A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency. Answer:

Defect clustering means a small number of modules often contain a large proportion of defects.

Historical defect data should be analyzed to identify modules with high defect density.

High-risk modules should receive deeper functional, integration, regression, and exploratory testing.

Low-defect modules can still be tested, but effort can be prioritized according to risk.

For example, if payment and authentication contain most defects, they should receive extra test cycles. This focused allocation improves defect detection while using limited testing time efficiently.

12. Question

A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution. Answer:

- The issue is a performance defect because response time becomes unacceptable under heavy load.
- It may also be classified as a scalability or capacity-related defect depending on the root cause.
- Load testing can measure response time, throughput, resource usage, and failure rate.
- Stress testing can determine how the application behaves beyond expected capacity.
- Correct classification helps assign the issue to the appropriate technical investigation.
- It also guides suitable fixes such as optimization, caching, database tuning, or scaling.

13. Question

A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.

Answer:

- Historical defect data identifies functions and modules that previously failed.
- Testers should select old defect cases and include them in the regression suite.
- Related areas should also receive additional tests because a previous defect may indicate wider risk.
- Frequently recurring and high-severity defects should receive higher regression priority.
- Automation can execute stable historical regression tests quickly after each release.
- This reuse improves coverage, reduces repeated defects, and saves testing time.

14. Question

During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.

Answer:

Testers should communicate requirements, risks, assumptions, and integration expectations with developers.

They should participate in requirement and design reviews before coding is completed.

Testers can identify interface mismatches, missing data rules, and unclear responsibilities early.

Regular discussions and shared defect information reduce misunderstandings between teams.

Early collaboration allows integration test conditions to be planned before modules are combined. This reduces integration defects and supports faster, more reliable development.

15. Question

A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality. Answer:

- Recurring defects should be analyzed to find their root causes rather than only fixing symptoms.
- Use techniques such as root cause analysis, defect trend analysis, and process reviews.
- Update coding standards, requirements templates, checklists, and test cases based on findings.
- Conduct reviews or training when repeated defects result from misunderstanding or process gaps.
- Add regression tests for previously fixed defects so they do not return in later releases.
- Defect prevention reduces recurrence and improves the development and testing process over time.

16. Question

A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability. Answer:

- Create a common test strategy defining scope, environments, test levels, roles, and quality criteria.
- Use standardized test case templates and consistent expected-result formats.
- Cover functional, usability, compatibility, performance, security, and crash-related scenarios.
- Maintain a regression suite and execute it after significant changes to the application.
- Record defects with severity, priority, steps, logs, and device information.
- Standardized testing gives repeatable results and improves mobile application reliability.

17. Question

A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.

Answer:

- Testing is the process of finding defects by evaluating software against expected behavior.
- Debugging is the developer's activity of locating, analyzing, and fixing the cause of a defect.
- For example, a tester finds that an ATM deducts the wrong amount during withdrawal.
- The tester reports the failure with steps and evidence, while the developer investigates the code.
- After the fix, the tester retests the function and performs regression testing.
- Testing detects problems, while debugging identifies and removes their underlying causes.

18. Question

An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled. Answer:

- Create a browser and device matrix based on the browsers most used by target customers.
- Test core functions such as login, search, cart, checkout, payment, and page layout on each browser.
- Include different operating systems, screen sizes, and supported browser versions where required.
- Use compatibility testing tools and real devices or environments for important combinations.
- Record browser-specific failures with screenshots, versions, steps, and severity.

- Regular cross-browser regression testing prevents inconsistent behavior after future changes.

19. Question

A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.

Answer:

Validation should verify that inputs, business rules, calculations, and outputs satisfy requirements.

Testers should identify missing checks for invalid, incomplete, duplicate, and boundary inputs.

For logistics, cases such as invalid addresses, negative quantities, and incorrect shipment status should be tested.

Expected results must be clearly defined so inconsistent outputs can be detected.

Validation tests should be performed at unit, integration, and system levels where appropriate. Proper validation testing improves data accuracy, consistency, and overall system reliability.

20. Question

A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.

Answer:

- Testing as an engineering activity requires planning, measurable objectives, and systematic test design.
- Identify requirements, risks, inputs, outputs, and expected results before executing tests.
- Use techniques such as equivalence partitioning, boundary values, decision tables, and state testing.
- Review test cases for coverage and trace them to requirements.
- Prioritize critical and high-risk scenarios instead of depending on random execution.
- Systematic design provides repeatable coverage and increases the chance of finding important defects.

21. Question

A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.

Answer:

The issue should be investigated by tracing data flow and interfaces between connected modules.

Check whether field formats, validation rules, mappings, and update transactions are consistent.

Integration tests should verify that data sent by one module is correctly received and processed by another.

Negative and boundary cases should test missing, invalid, duplicate, and conflicting data.

Defects should be recorded with the source module, target module, data values, and expected result.

Thorough integration and data validation testing helps maintain consistency across the system.

22. Question

A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.

Answer:

- In requirements, testers review clarity, completeness, risks, and testability.

During design, they identify integration points, data conditions, and possible failure scenarios.
During development, they prepare test cases and support developers with testable acceptance criteria.
During integration and system testing, they execute tests, report defects, and verify fixes.
During maintenance, they perform regression testing and analyze production defects.
Early tester involvement prevents misunderstandings and reduces costly late-stage changes.

23. Question

A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.

Answer:

- The main gap is insufficient performance and load-test coverage for peak usage conditions.
- Identify realistic user volumes, concurrent sessions, transaction rates, and resource limits.
- Use load, stress, and endurance tests to evaluate behavior under different conditions.
- Include critical business transactions rather than testing only simple user actions.
- Monitor response time, throughput, CPU, memory, database, and error rates during tests.
- Better scenario coverage reveals capacity problems before deployment and prevents peak-load failures.

24. Question

A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.

Answer:

- Create a centralized defect repository with a unique ID for every reported defect.
- Record summary, module, environment, steps to reproduce, expected result, actual result, severity, and priority.
- Track ownership, status, fix details, retest results, and closure information.
- Use the repository to generate defect trends by module, type, severity, and release.
- Historical records help identify recurring problems and improve future test design.
- Good documentation improves traceability, communication, accountability, and defect analysis.

25. Question

A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.

Answer:

Start with risk-based planning because transaction processing is a critical banking function.
Create detailed test cases for deposits, withdrawals, transfers, reversals, limits, and failed transactions.
Use boundary, negative, integration, security, and regression testing along with normal functional tests.
Maintain traceability from banking requirements to test cases and defects.
Automate stable regression and transaction tests to improve repeatability.
Structured testing increases coverage and reliability while reducing repeated transaction failures.

26. Question

A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.

Answer:

- Testing should include normal, invalid, boundary, and extreme conditions instead of only typical inputs.
- Identify minimum and maximum values from each requirement and business rule.
- Test exact boundaries and values immediately below and above them.
- For example, if a quantity allows 1 to 100, test 0, 1, 2, 99, 100, and 101.
- Boundary Value Analysis is effective because defects commonly occur around limits.
- This approach exposes edge-case failures early and improves overall input coverage.

27. Question

A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.

Answer:

Use a common test process covering planning, design, execution, defect management, and reporting.

Define standard templates for test cases, test data, defect reports, and completion criteria.

Apply common test design techniques and review practices across all modules.

Use shared quality metrics such as coverage, pass rate, defect density, and leakage.

Conduct peer reviews and audits to ensure the process is followed consistently.

Standardization makes testing repeatable and produces more uniform quality across modules.

28. Question

A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.

- Answer:
- Identify all input fields and define valid, invalid, missing, unexpected, and boundary values.
 - Test special characters, incorrect formats, empty values, oversized values, and unexpected data types.
 - Verify that invalid input produces a safe and meaningful error instead of a crash.
 - Use equivalence partitioning and boundary value analysis to reduce gaps in test coverage.
 - Include security-related validation for inputs where injection or malicious data is possible.
 - Strong input validation testing makes the application more robust and resistant to unexpected conditions.

29. Question

A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.

Answer:

Analyze escaped defects by module, severity, cause, test phase, and missed scenario.

Check whether requirements, test coverage, environments, data, or regression testing were inadequate.

Add tests for production defects and related scenarios to the regression suite.

Use risk-based testing to give more attention to critical and frequently failing areas.

Improve reviews, automation, environment consistency, and entry/exit criteria where needed.

Continuous analysis and improved coverage reduce defect leakage and strengthen release quality.

30. Question

A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency. Answer:

- Adopt early testing by involving testers during requirements and design reviews.
- Use risk-based planning to identify high-impact areas and test them first.
- Apply systematic test design with equivalence classes, boundaries, decision tables, and negative cases.
- Automate suitable unit and regression tests to provide fast feedback after changes.
- Track defects to their root causes and use the results to improve development and testing practices.
- Finding defects early reduces rework, lowers cost, shortens delivery time, and improves software quality.